

Jev serving topology

How the Jev playground's three objects (State, Question, Response) move through hardware and software so that millions of run requests share the load without a single choke point.

— Push state (PUT, stored on NVMe) — Run question (POST, pinned to a state position) — Response token stream — Cache miss read

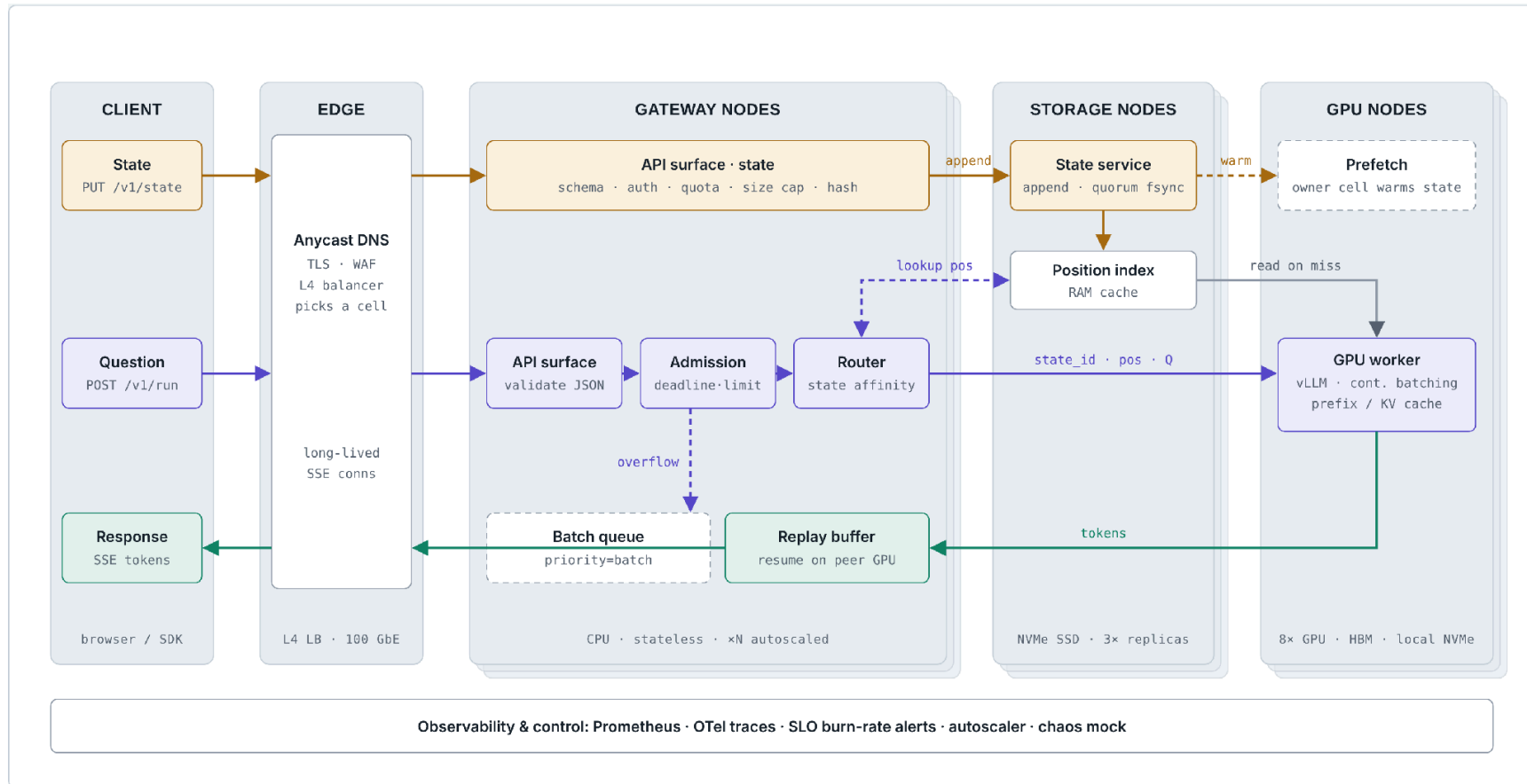


Figure 1. One Jev cell. Each column is a hardware tier and the stacked edges mean it scales out by adding nodes. State is pushed once and lives on replicated NVMe; each run request carries only a pointer (*state_id* plus position) and its JSON question, so the router can send it to the GPU cell that already has that state warm. To serve more traffic, add cells behind the same edge.

The two calls

State is written once and read many times. A run request never re-sends the context; it points at an exact, immutable position in it.

PUT Push state

```
PUT /v1/state/ctx_8f2a
Idempotency-Key: 3c1e...
{ "schema": "jev.state.v1",
  "data": { ... context ... } }

→ 201 { "state_id": "ctx_8f2a",
        "position": 42,
        "sha256": "9b0d..." }
```

- Validated at the API surface: schema version, auth, quota, size cap.
- State is append-only. Every write returns a new monotonic position; older positions never change.
- Ack only after 2 of 3 NVMe replicas fsync (quorum), so a lost disk loses nothing.
- Same content hash = no second write (dedupe).

POST Run question

```
POST /v1/run
X-Deadline-Ms: 30000
{ "state_id": "ctx_8f2a",
  "position": 42,
  "question": { "q": "...", "params": {...} },
  "priority": "interactive" }

→ 200 text/event-stream
```

- Rejected with 409 if position is past the committed head, 404 if the state is unknown.
- Pinning to a position makes every answer reproducible, even after the state grows.
- KV-cache key = hash(state_id, position); the next question on the same position skips the prefill.

Where it would choke, and what stops it

CHOKE POINT	FIX IN THIS DESIGN	BULWARK STAGE
Context re-sent and re-processed on every question	Push state once; run requests carry a few hundred bytes. State-affinity routing plus prefix/KV cache means the GPU prefills a state once, not per question.	Router
One storage box holding all state	Consistent-hash shards by state_id across NVMe nodes, 3× replicas. Adding a node moves only 1/N of the keys.	State tier
Gateway CPU or connection count	Gateways keep no state, so the autoscaler adds pods on CPU and open-stream count. The edge L4 balancer spreads connections.	API surface
Traffic spike piles up queues until everything times out	Deadline propagation, priority classes and a gradient-based adaptive limit. Fail fast with 429 and Retry-After; batch work drains to its own queue.	Admission
A slow or dead GPU node	Health score, circuit breaker, power-of-two choices, hedged request after p95 TTFT.	Router

CHOKE POINT	FIX IN THIS DESIGN	BULWARK STAGE
GPU dies after 200 tokens were already sent	Replay buffer resumes the stream on a peer with the same seed, so the client never sees a broken response.	Replay buffer
Hot state (one context used by many users)	Prefetch replicates hot positions to several GPU cells; the router spreads load across those owners.	Router
One bad deploy or region takes everything down	Cell architecture: each cell is a full copy of the stack with its own blast radius. The edge drains a failing cell.	Ops loop

Sizing one deployment

The GPUs are the real limit. Every other tier is cheap to scale, so the design spends its effort keeping GPU work to the minimum: no repeated prefill, no wasted retries, no queue collapse.

TIER	FORMULA	EXAMPLE AT 10,000 RUN REQ/S ($\approx$ 860 M/DAY)
Gateway	peak rps $\div$ rps per node, plus N+2 spare. Concurrent streams = rps $\times$ stream length.	10k $\div$ 5k = 2 $\rightarrow$ run 6 nodes $\cdot$ 80k open streams
Storage	state writes/s $\times$ avg size $\times$ 3 replicas, kept under 50% of disk bandwidth	1k/s $\times$ 200 KB $\times$ 3 = 600 MB/s $\rightarrow$ 6 NVMe nodes
GPU	rps $\times$ output tokens $\div$ tokens/s per node $\div$ 0.7 target utilisation	10k $\times$ 250 $\div$ 25k $\div$ 0.7 $\approx$ 143 nodes

Per-node throughput figures are placeholder assumptions. Replace them with load-test numbers from your own model and hardware before you trust the totals; GPU tokens per second depends heavily on model size and batch settings.

SLOs for Jev

SLI	TARGET (30-DAY WINDOW)	MEASURED AT
Run availability (non-5xx, stream completed)	99.5%	Edge / ingress, so a dead replica can't hide
Time to first token, interactive	99% < 1 s	Gateway, histogram with buckets near 1 s
State write latency (quorum ack)	99% < 250 ms	State service
Mid-stream failover success	99% resumed	Replay buffer counter

Alerting reuses the project's multiwindow burn-rate rules: 14.4 $\times$ over 1 h and 5 m pages, 6 $\times$ over 6 h and 30 m opens a ticket. A 429 from admission counts as shed load, not as an error.